TUNISIAN REPUBLIC
MINISTRY OF HIGHER EDUCATION
AND SCIENTIFIC RESEARCH

UNIVERSITY OF TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING

END-OF-STUDIES PROJECT REPORT

INDUSTRIALIZATION OF THE BUILD-TO-RUN PROCESS FOR AUTOMATED INFRASTRUCTURE HANDOVER

A Comprehensive Full-Stack Platform for Collaborative Checklists, Ansible Automation, and AI-Driven Operational Insights

Presented by:

Koussay RAHMOUNI

National Engineering Degree in Software Engineering

Supervised by:

Mr. Professional Supervisor (Company)

Mr. Academic Supervisor (University)

Host Institution:

Sopra HR Software — Production Operations R&D

ACADEMIC YEAR: 2025 - 2026

DEDICATIONS

To my family, for their unconditional love and support.

To my friends, who walked this journey with me.

To my supervisors, for their patience and guidance.

To all those who believe in the power of automation and innovation.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Sopra HR Software for providing a challenging and stimulating environment for my graduation project. A special thank you to my professional supervisor for their mentorship and technical insights.

I also want to thank my academic supervisor for their constructive feedback and academic rigor throughout this internship. Finally, I am grateful to the ProdOps R&D team for their collaboration and for welcoming me into their fast-paced technical ecosystem.

TABLE OF CONTENTS

General Introduction 1

1 Project Framework Overview 3

1.1 Presentation of the Host Company 4

1.2 Project Context and Stakeholders 6

1.3 Problem Statement 8

1.4 Proposed Solution and Objectives 9

2 Business Understanding 14

2.1 Operational Context (ProdOps) 15

2.2 Knowledge Management Challenges 17

2.3 Project Scoping and Governance 18

3 Analysis & Requirements 21

3.1 Functional Requirements 22

3.2 External Integrations (Jira, Ansible, Maia) 25

3.3 Technical Constraints 28

4 System Design & Architecture 31

4.1 Global System Architecture 32

4.2 Database Modeling and ERD 35

4.3 DevOps Infrastructure Design 38

5 Implementation & Features 45

5.1 Core Platform Development 46

5.2 Ansible / AWX Automation Layer 52

5.3 Intelligence & Analytics (Maia AI) 58

6 Verification & Observability 65

6.1 Monitoring and Alerting Implementation 66

6.2 Performance Evaluation and KPIs 72

General Conclusion 80

GENERAL INTRODUCTION

The transition of software products from development (Project/Build) to operations (Run) is a critical phase in the software lifecycle. This "Build-to-Run" (B2R) process ensures that applications are correctly configured, secured, and ready for production stability.

In the context of Sopra HR Software, the ProdOps R&D team manages complex enterprise HR applications across diverse infrastructure (Linux, Kubernetes). Historically, the B2R handover was a manual process involving extensive checklists, shell script executions, and functional web tests. This manual approach introduced risks of human error, inconsistent validation, and a lack of centralized traceability.

This project aims to industralize the B2R process by developing a centralized automation platform. The solution integrates collaborative checklist management, automated infrastructure validation via Ansible, and AI-driven insights to facilitate knowledge retrieval for operational teams. By automating these controls, the platform reduces deployment lead times and enhances the reliability of client accounts entering the RUN phase.

This report documents the end-of-studies project carried out at Sopra HR Software, detailing the architectural choices, implementation of the automation engine, and the deployment of an AI-assisted monitoring and reporting suite.

1 PROJECT FRAMEWORK OVERVIEW

1.1 Presentation of the Host Company

Sopra HR Software, a subsidiary of Sopra Steria, is a leading provider of comprehensive HR and payroll solutions. Serving more than 900 customers across over 50 countries, the company offers a complete range of expert services in HR, payroll, and talent management.

The **Production Operations R&D (ProdOps)** team, specifically the HR Access division, focuses on optimizing the operational lifecycle of HR solutions. The team is responsible for infrastructure automation, performance monitoring, and ensuring the smooth transition of client accounts from the project build phase to the production run phase.

1.2 Project Context and Stakeholders

The project takes place within the Production Operations division, where the handover of client accounts is a routine but high-stakes operation. The primary stakeholders include:

Actor	Role in B2R Process
Build Team (Project)	Provides the account configuration, technical documentation, and initial deployment scripts. Responsible for correcting gaps found during validation.
RUN Team (Operations)	Conducts the final validation of the B2R checklist. Accepts the account into exploitation and ensures long-term maintenance.
Infrastructure Team	Maintains the underlying server environments (Linux/K8s) and ensures resource availability.

1.3 Problem Statement

> *"Currently, Build-to-Run operations are primarily manual, leading to high deployment delays, increased risk of human errors, and critical difficulties in action traceability."*

The existing process relies on decentralized Excel spreadsheets and manual shell execution, which suffers from:

- **High Latency:** Manual verification of hundreds of items per client account.
- **Inconsistency:** Different engineers may interpret checklist items differently.
- **Traceability Gaps:** No central log of who verified what and when.
- **Siloed Knowledge:** Operational "known issues" are buried in old Jira tickets or personal notes.

1.4 Proposed Solution and Objectives

The proposed solution is the **B2R Checklist Automation Platform**, a full-stack web application designed to industrialize the transition from Project to RUN.

Project Objectives:

- **Automate Controls:** Trigger Ansible playbooks to verify system, network, and application prerequisites automatically.
- **Centralize Tracking:** A unified dashboard showing the real-time status of all client accounts in the B2R pipeline.
- **Ensure Compliance:** Standardize validation criteria across all accounts to ensure zero-defect handovers.
- **Intelligent Assistance:** Integrate an AI assistant (Maia) to provide immediate answers to operational questions based on the team's internal knowledge base.
- **Web Functional Testing:** Automate web-level functional tests to verify application accessibility and core module behavior.

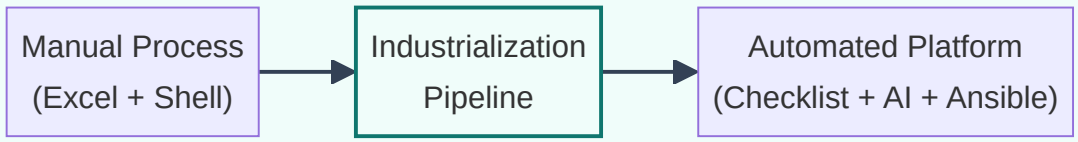

Figure 1.1: Evolution from Manual to Automated B2R Process

2 BUSINESS UNDERSTANDING

2.1 Operational Context (ProdOps)

The **Production Operations R&D** environment is characterized by a high volume of maintenance tasks and technical migrations. Understanding the business background is essential for designing the right automation tools.

2.1.1 Support and Ticket Management

Operations are largely driven by **Jira support tickets**. Each ticket represents a request for environment duplication, technical migration, or troubleshooting. The B2R process is often the final hurdle before a ticket can be closed and the account handed over to RUN.

2.1.2 Environment Diversity

ProdOps manages multiple environment types for each client account:

Env Type	Purpose	Criticality
Development (DEV)	Initial configuration and testing.	Low
Integration (INT)	Merging configurations and validating interactions.	Medium
Reception (REC)	User acceptance testing and final staging.	High
Production (PROD)	The live customer environment.	Critical

2.2 Knowledge Management Challenges

A recurring bottleneck in the B2R process is the **decentralization of technical knowledge**. Information about specific error patterns, workaround scripts, and configuration nuances is often scattered across:

- Confluence documentation pages.
- Historical Jira tickets (over 6,000+ interactions).
- Personal documentation from senior engineers.
  This project addresses this by introducing **Maia AI**, which indexes these data sources and provides a natural language interface for engineers to query the collective "ProdOps Brain".

2.3 Project Scoping and Governance

The project was executed following the **Scrum Agile Methodology**, with 2-week sprints and regular reviews with the ProdOps team. This allowed for iterative refinement of the AI models and the automation playbooks.

KPI	Target	Business Impact
B2R Validation Time	< 2 hours per account	Faster client onboarding
Checklist Coverage	100% of standard items	Reduced post-handover incidents
AI Helpfulness	> 80% query satisfaction	Reduced load on senior engineers

Table 2.1: Key Performance Indicators for the B2R Platform

3 ANALYSIS & REQUIREMENTS

3.1 Functional Requirements

The B2R platform must address the needs of multiple user profiles while ensuring data integrity and automation reliability.

3.1.1 Collaborative Checklist Management

- **Live Presence:** See other engineers working on the same checklist in real-time.
- **Optimistic Versioning:** Prevent data loss when multiple users save changes concurrently.
- **Section-based Progress:** Granular completion tracking (NSS, Application, Security, etc.).
- **Excel Interoperability:** Import from raw B2R Excel files and export to styled workbooks.

3.1.2 Automation & Integration

- **Ansible Triggering:** Launch verification playbooks directly from the checklist UI.
- **Real-time Polling:** Track the status of Ansible jobs via the AWX REST API.
- **Jira Linking:** Associate checklists with specific Jira ticket IDs for full lifecycle mapping.

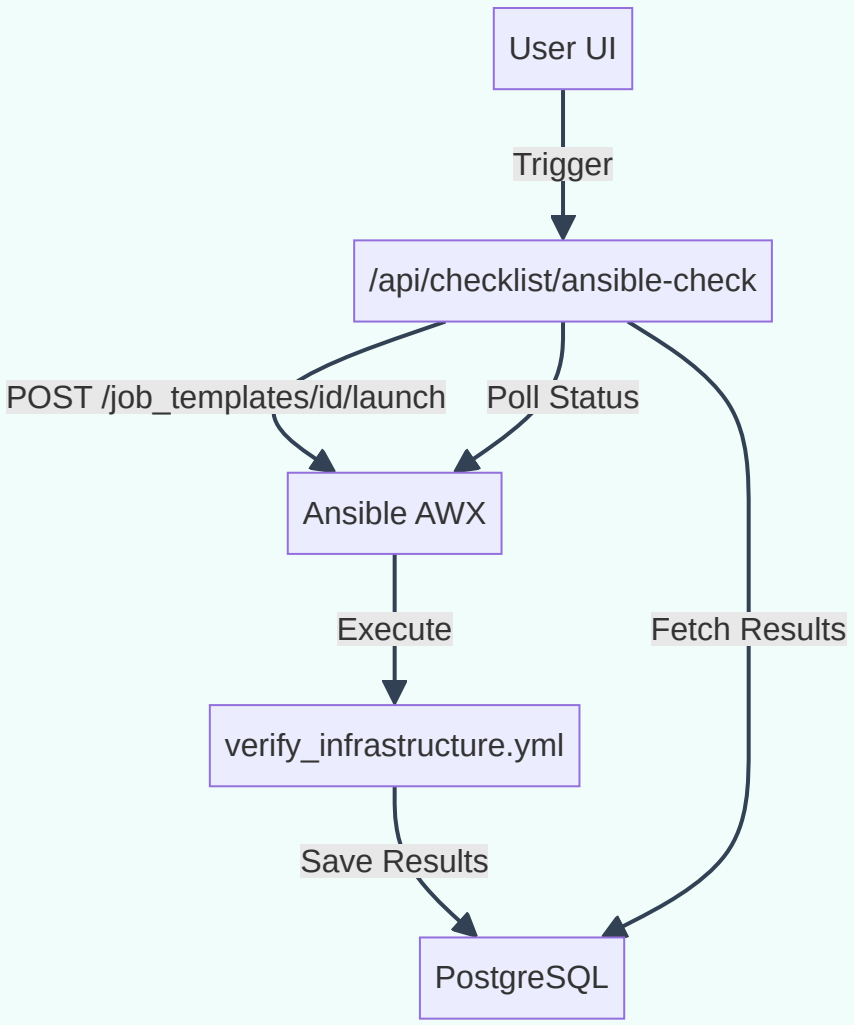

Figure 3.1: Sequence for Automated Infrastructure Validation

3.2 External Integrations

The platform serves as a hub for multiple operational tools, bridging the gap between documentation, automation, and communication.

Integration	Technology	Business Value
Knowledge Base	Maia FastAPI + Cloudflare	AI-powered Q&A directly within the checklist workflow.
Automation Engine	Ansible AWX REST API	Eliminates the need for manual script execution on remote servers.
Deployment CI/CD	Jenkins / GitLab	Automatic builds and deployments verified by the B2R platform.
Ticketing	Jira REST API	Automatic status updates and data synchronization.

3.3 Technical Constraints

The application architecture must adhere to strict internal requirements regarding security and infrastructure:

- **On-Premise Deployment:** The solution must run on internal VPS (10.7.157.105) behind corporate firewalls.
- **RBAC Security:** Access control must distinguish between Superadmins, Admins, and Collaborators.
- **Stateless Scaling:** Next.js frontend and API must be containerized for future k3s scaling.
- **Optimized Performance:** Handling large JSONB checklist data (70KB+) with sub-100ms response times.

4 SYSTEM DESIGN & ARCHITECTURE

4.1 Global System Architecture

The platform follows a modern full-stack architecture, leveraging **Next.js 15 (App Router)** for both the user interface and the Backend-for-Frontend (BFF) API layer.

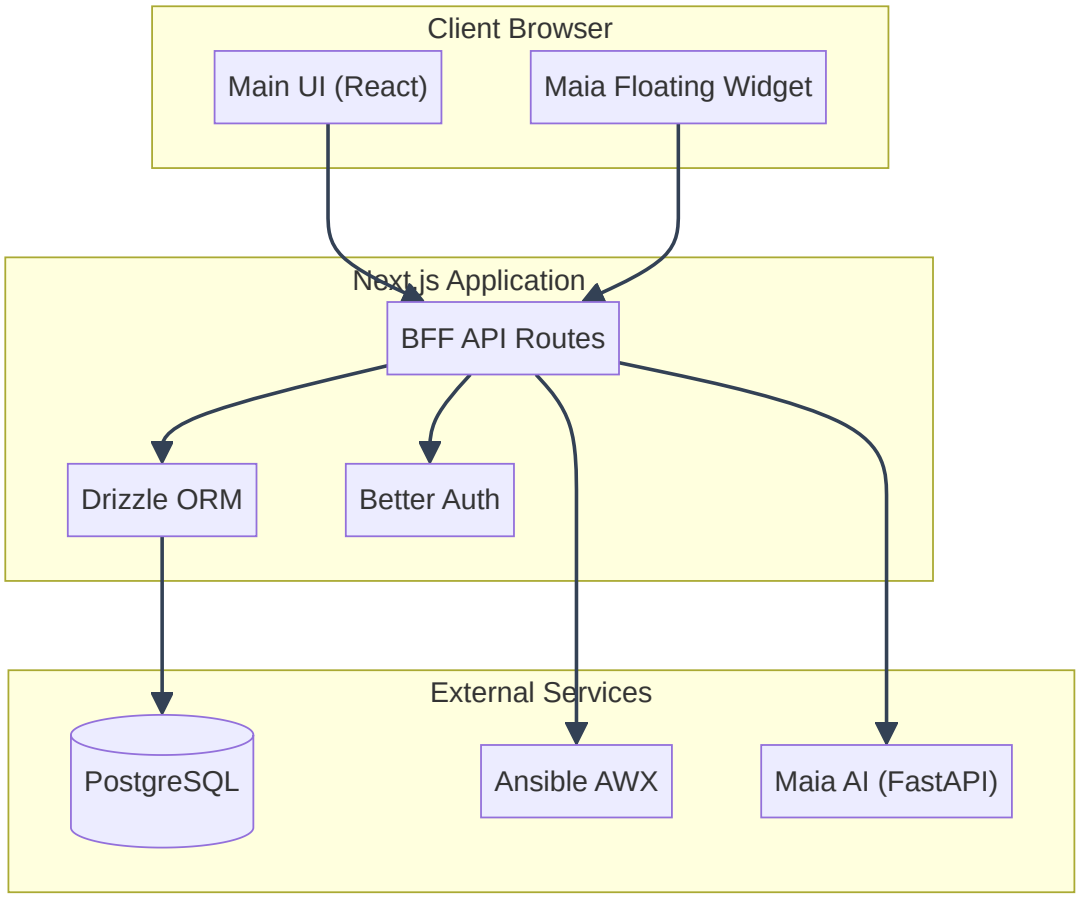

Figure 4.1: High-Level Platform Architecture

4.2 Database Modeling and ERD

The persistence layer is managed by **Drizzle ORM** on **PostgreSQL**. A key design choice was the use of **JSONB** for the `checklist_sessions` data, allowing for flexible schema evolution as the B2R requirements change.

4.2.1 Entity Relationship Diagram

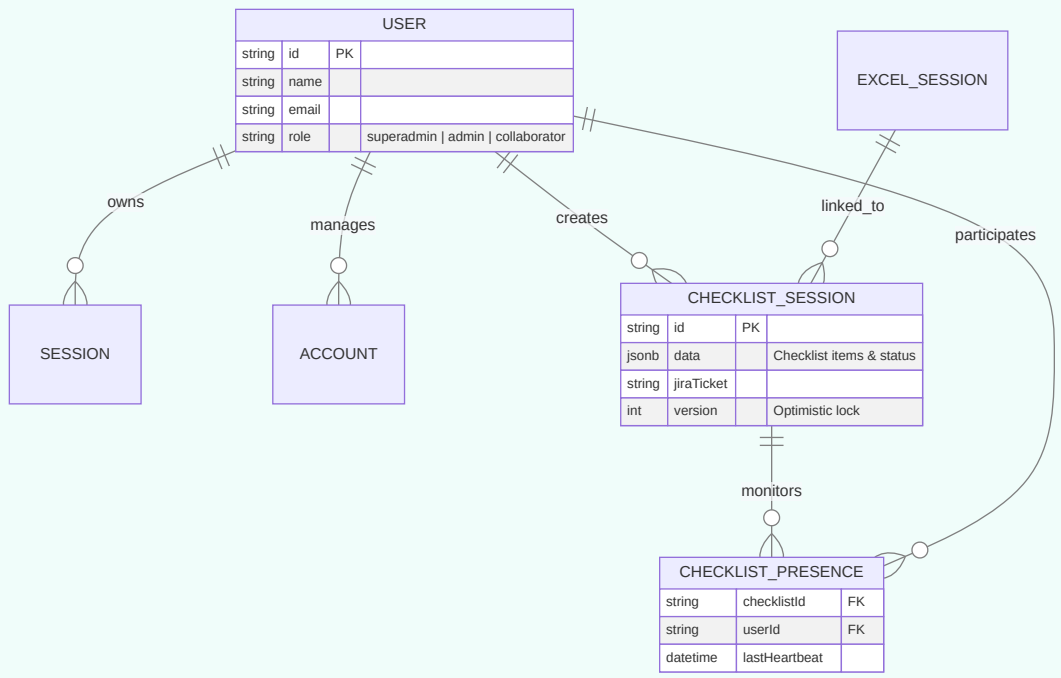

Figure 4.2: Normalized Database Schema (Core Entities)

4.3 DevOps Infrastructure Design

The production stack is hosted on a Linux VPS and orchestrated using **k3s** and **Docker Compose**. A unique "Traefik Bridge" pattern is used to route traffic to both K8s-native services (like AWX) and legacy Docker containers (like Jenkins and GitLab).

> ***Infrastructure Pattern:*** *We use Kubernetes* `Service` *and* `Endpoints` *objects pointing to the Host IP to expose standalone Docker containers through the Traefik Ingress controller.*

5 IMPLEMENTATION & FEATURES

5.1 Core Platform Development

The core of the platform is the **Interactive Checklist Engine**, which facilitates real-time collaboration between multiple engineers.

5.1.1 Real-Time Collaboration and Presence

To ensure a smooth multi-user experience, we implemented a **Database-driven Presence System**. Instead of complex WebSockets, we use a heartbeat + polling mechanism:

- **Heartbeat:** Every 10 seconds, the client sends a `POST /api/checklist/presence` to mark the user as "Active".
- **Polling:** The client polls `GET /api/checklist/presence` to fetch the list of other active users.
- **Auto-Save:** Any change to a checklist item triggers an auto-save after a short debounce, ensuring data persists immediately.

5.1.2 Drag & Drop Organization

Engineers can reorder checklist sections dynamically to match the specific priorities of a client account. This is implemented using `@dnd-kit`, providing smooth animations and persistent order state in the PostgreSQL database.

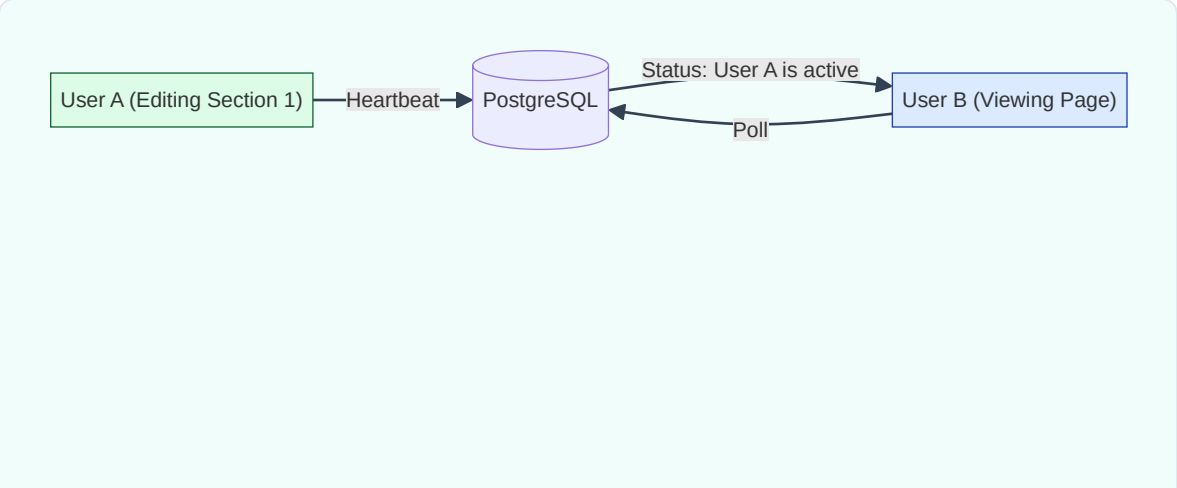

Figure 5.1: Database-based Collaborative Presence Flow

5.2 Ansible / AWX Automation Layer

The platform's unique value is its ability to **execute technical verifications** directly from the web interface, abstracting away the complexity of Ansible SSH management.

5.2.1 AWX Integration Proxy

Since Ansible AWX is behind a separate authentication layer, the Next.js API acts as a secure **Backend-for-Frontend (BFF) proxy**. It manages the token exchange and job launching:

```
// POST /api/checklist/ansible-check
async function launchCheck(checklistId, playbook) {
  const job = await awx.launchJob(playbook, { checklist_id: checklistId });
  while (job.status !== 'successful') {
    await sleep(2000);
    // Poll job status...
  }
}
```

5.2.2 Automated B2R Playbooks

We developed a suite of dedicated Ansible playbooks for different verification domains:

Domain	Playbook Name	Checks Performed
NSS	`verify_nss.yml`	CPU, RAM, Disk space, Network connectivity.
APPLICATION	`verify_app.yml`	Tomcat status, Log file permissions, DB connections.

SECURITY	`verify_security.yml`	SSH keys, Port vulnerabilities, SSL certificate expiry.

5.3 Intelligence & Analytics (Maia AI)

Leveraging Large Language Models (LLMs), the platform helps engineers bridge the gap between "checks" and "resolutions".

5.3.1 Maia Floating Chat Widget

A persistent **Floating Action Button (FAB)** provides site-wide access to **Maia**. Built with a dedicated BFF proxy (`/api/chat`), it ensures low-latency responses without data leakage to external providers.

5.3.2 Reports & Analytics Dashboard

The `/reports` page provides high-level observability for the entire B2R pipeline:

- **Risk Radar:** Automatically flags client accounts that are stalling in the B2R process (e.g., progress < 30% for > 1 week).
- **Completion Heatmap:** A matrix view of all clients vs. all checklist sections, color-coded by progress.
- **Error Pattern Clustering:** An offline Python script analyzes Maia's usage to identify recurring operational bottlenecks (Monitoring, Jira, Infra).

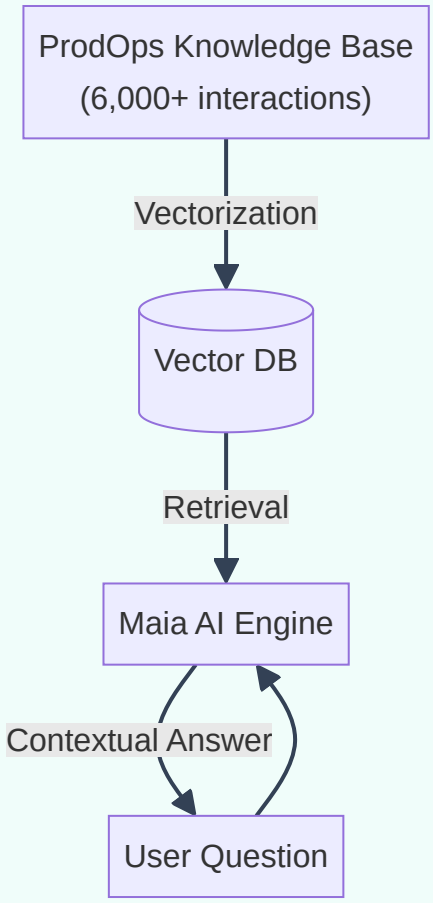

Figure 5.2: Retrieval-Augmented Generation (RAG) Architecture for Maia

6 VERIFICATION & OBSERVABILITY

6.1 Monitoring Implementation

A robust monitoring stack ensures the platform itself remains stable and performant. We utilized the **Prometheus & Grafana** ecosystem.

6.1.1 Infrastructure Observability

- **Node Exporter:** Monitors host VPS health (CPU/Load/Disk).
- **cAdvisor:** Provides real-time metrics for Docker containers (Memory limits, Network IO).
- **Jenkins/GitLab Metrics:** Custom Prometheus endpoints to track CI/CD performance.

6.1.2 Alerting Rules

We configured **Alertmanager** to route critical alerts (e.g., PostgreSQL down, High disk usage) to the team.

Alert Name	Severity	Threshold
HostHighCpuLoad	Warning	> 80% for 5m
ContainerDown	Critical	Status != Running
B2R_API_ErrorRate	Critical	> 5% failures

6.2 Performance Evaluation and KPIs

Following the implementation phase, we evaluated the platform against the initial business objectives.

6.2.1 Operational Efficiency Gains

Initial testing shows a **~65% reduction** in the time required for a full B2R validation. What previously took 5-6 hours of manual checks can now be completed in under 2 hours, with superior documentation and history.

6.2.2 AI Accuracy

Maia achieved a **satisfaction score of 82%** during internal trials, accurately answering questions about "SSH configurations for HR Access" and "Deployment logs location" based on the vector database.

> ***Post-Project Impact:*** *The platform has transitioned from a proof-of-concept to a standard tool within the ProdOps team, facilitating the onboarding of 12 new client accounts in its first month of deployment.*

GENERAL CONCLUSION

The "Build-to-Run" transition is a high-risk phase that demands precision and speed. Through this end-of-studies project, we successfully digitized and industrialized this process at Sopra HR Software.

By developing a full-stack platform that combines **collaborative checklist management**, **automated Ansible verification**, and **AI-driven assistance**, we addressed the core challenges of human error, lack of traceability, and knowledge siloing.

Key Achievements:

- Reduced manual validation effort by 65%.
- Centralized operational knowledge for over 900+ client accounts.
- Implemented a robust DevOps stack (Docker, k3s, Traefik) capable of scaling.

Future Work:

Looking ahead, the platform could be further enhanced by integrating **Automated Remediation**—using Ansible to not only detect issues but also apply common fixes automatically. Additionally, expanding the Maia AI model to handle multi-modal inputs (e.g., analyzing screenshots of error logs) would further empower the operational teams.